

PRACTICAL 6 : Docker Compose for Multi-Container Applications.

- Create a docker-compose.yml file for a multi-container setup (e.g., Flask/FastAPI app + PostgreSQL database).
- Run the application using Docker Compose and test the communication between services.

To create and run a multi-container application using Docker Compose with:

- Flask Application
 - PostgreSQL Database
-

Requirements

- Docker Desktop
 - VS Code
 - Python 3.x
 - Internet connection (for downloading Docker images)
-

Project Structure

```
FlaskComposeProject/  
├── app.py  
├── Dockerfile  
├── docker-compose.yml  
├── requirements.txt  
└── .env
```

Step 1: Create Flask Application (app.py)

```
from flask import Flask  
import psycopg2  
import os  
  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    try:  
        conn = psycopg2.connect(  
            host=os.getenv("DB_HOST"),  
            database=os.getenv("DB_NAME"),
```

```
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )

    conn.close()

    return "Connected Successfully to PostgreSQL!"

except Exception as e:
    return "Database Connection Failed: " + str(e)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 2: Create requirements.txt

```
Flask
psycopg2-binary
```

Step 3: Create Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Step 4: Create .env File

```
DB_HOST=db
DB_NAME=mydatabase
DB_USER=postgres
DB_PASSWORD=postgres
```

Step 5: Create docker-compose.yml

```
version: "3.9"
```

```
services:

  web:
    build: .
    container_name: flask_app

    ports:
      - "5000:5000"

    depends_on:
      - db

    environment:
      DB_HOST: db
      DB_NAME: mydatabase
      DB_USER: postgres
      DB_PASSWORD: postgres

  db:
    image: postgres:16

    container_name: postgres_db

    restart: always

    environment:
      POSTGRES_DB: mydatabase
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres

    ports:
      - "5432:5432"

    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Part (b): Run the Application Using Docker Compose

Step 1: Open Terminal

Navigate to the project folder.

```
cd FlaskComposeProject
```

Step 2: Build the Containers

```
docker compose build
```

If successful, Docker builds the Flask image.

Step 3: Start the Containers

```
docker compose up -d
```

Expected output:

```
Creating network "flaskcomposeproject_default"
Creating volume "postgres_data"
Creating postgres_db
Creating flask_app
```

Step 4: Verify Running Containers

Run:

```
docker ps
```

Expected output:

CONTAINER ID	IMAGE	STATUS
xxxxxxxx	flask_app	Up
xxxxxxxx	postgres:16	Up

Both containers should be running.

Step 5: Test Communication Between Services

Method 1 (Recommended)

Open your browser and visit:

```
http://localhost:5000
```

Expected Output:

```
Connected Successfully to PostgreSQL!
```

This confirms that:

- Flask is running.
 - Flask successfully connected to PostgreSQL using the Docker Compose network.
-

Method 2 (Check Flask Logs)

```
docker compose logs web
```

Expected output:

Running on http://0.0.0.0:5000

Method 3 (Check PostgreSQL Logs)

```
docker compose logs db
```

Expected output:

```
database system is ready to accept connections
```

Step 6: Stop the Application

```
docker compose down
```

Expected output:

```
Stopping flask_app
Stopping postgres_db
Removing network
```

Architecture Diagram

